

MOVIE BOOKING APPLICATION

Deliverables for Movie Booking Application :

User Stories

- **US_01 Registration & Login**
 - Implemented user registration with mandatory fields (First Name, Last Name, Email, Login Id, Password, Confirm Password, Contact Number).
 - Enforced uniqueness for Email and Login Id.
 - Password confirmation validation.
 - Login, logout, and password reset functionality.
 - Validation messages shown for all constraint violations.
- **US_02 View & Search Movies**
 - Users can view all released movies.
 - Search functionality implemented by movie name.
- **US_03 Book Tickets**
 - Ticket booking integrated with database table Tickets.
 - Movie Name and Theatre Name linked as foreign keys to Movie.
 - Booking captures Movie Name, Theatre Name, Number of Tickets, and Seat Numbers.
- **US_04 Admin – View & Update Tickets**
 - Admin can view booked tickets from Tickets.
 - Ticket availability calculated from Movie table.
 - Admin can update: SOLD OUT when tickets = 0 , BOOK ASAP and DELETE movie

Role-Based Authentication & Authorization

- Implemented **JWT authentication** for secure login and session management.
- Defined **two distinct roles**:
 - **USER** → can register, login, search movies, book tickets, and view their bookings.
 - **ADMIN** → can view all booked tickets, update ticket status, and manage movie/ticket availability.

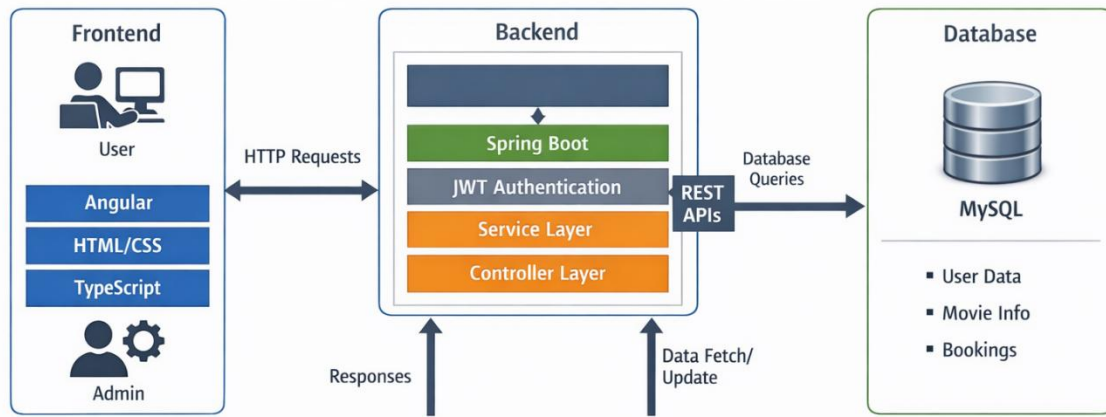
Backend (Spring Boot)

- Project structured under `com.moviebookingapp.*`.
- Constructor-based and setter-based dependency injection applied.
- ORM implemented with:
 - JpaRepository for SQL databases (MySQL).
 - Custom queries using `@Query`
- Implemented caching and session management.
- Unit tests for controller, service, and repository layers with >80% coverage.
- Global exception handling for error management.
- Logging and monitoring integrated for troubleshooting.
- Simple trigger added: when tickets = 0, status auto-set to `SOLD_OUT`.
- Integrated **Project Lombok** to reduce boilerplate code.
- Added **mappers** for converting between entities and DTOs.

Frontend (Angular)

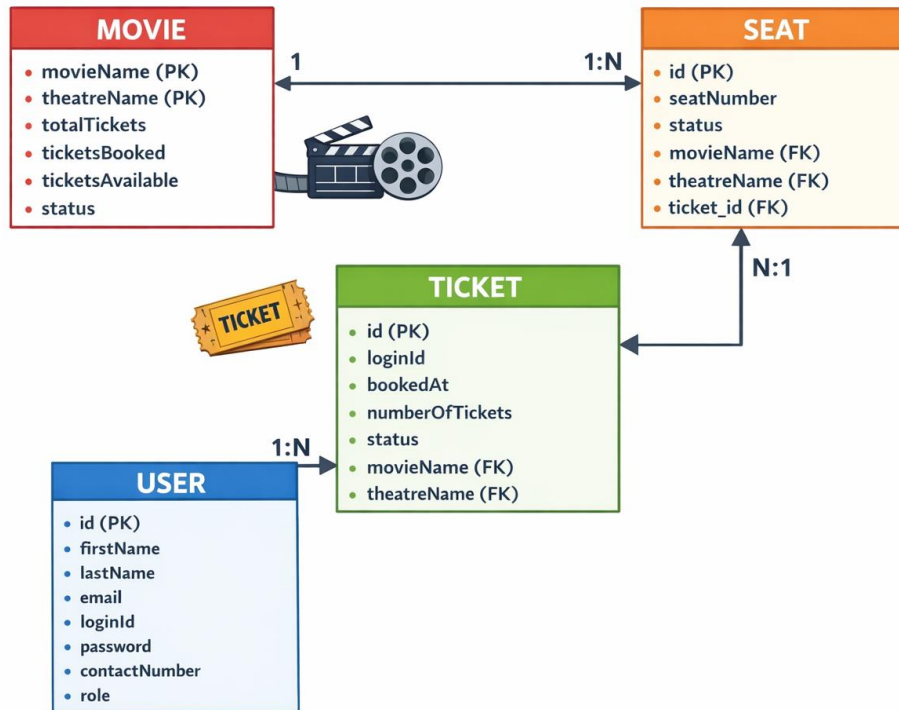
- Developed UI for all user stories.
- Implemented validation rules (required fields, unique checks, password match, etc.).
- Clear error/success messages with proper UI feedback.
- Consistent naming conventions and folder structures.
- Unit testing performed for components and services.

Movie Booking Application Architecture



Architecture Diagram

Basic ER Diagram for Movie Booking Application



Backend Api's

1. Registration API

Url : <http://localhost:8080/api/v1.0/moviebooking/register>

RequestPayload:

```
{
  "firstName": "shiva",
  "lastName": "rangu",
  "email": "shiva@gmail.com",
  "loginId": "123456789121",
  "password": "Shiv@1234",
  "confirmPassword": "Shiv@1234",
  "contactNumber": "9876543210"
}
```

Response :

```
{
  "success": true,
  "message": "User Created Successfully with Login Id 1234567s89121"
}

{
  "success": false,
  "message": "Email already exists"
}
```

2. Login API

Url : <http://localhost:8080/api/v1.0/moviebooking/login>

Request Payload:

```
{  
    "loginId": "1234567890",  
    "password": "Shiv@1234"  
}
```

Response :

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwicm9sZSI6I  
lJPTeVfVVNFUiIsImhhdCI6MTc3NDI2OTM0NSwiZXhwIjoxNzc0MjcyOTQ  
1fQ.HILfy1ENCZvJU3Pc4QC3M2Ajysnv1aKKUabm_ET5tBI
```

3. Forgot Password :

Url : <http://localhost:8080/api/v1.0/moviebooking/forgot>

Request Payload

```
{  
    "username": "Shiva@gmail.com",  
    "newPassword": "Shiv@123456",  
    "confirmPassword": "Shiv@123456"  
}
```

Response :

```
{  
    "success": true,  
    "message": "Password Updated"  
}
```

4. Reset Password

Url : <http://localhost:8080/api/v1.0/moviebooking/reset>

Request Payload :

```
{
  "newPassword": "Shiv@12345",
  "confirmPassword": "Shiv@12345"
}
```

Response :

```
{
  "success": false,
  "message": "New password must not be same as old password"
}
```

5. Upload Movie

Url : <http://localhost:8080/api/v1.0/moviebooking/add>

Request Payload :

```
{
  "movieName": "Mass",
  "theatreName": "PVR",
  "totalTickets": 20
}
```

Response

Movie Added Successfully with Seats

6. Delete Movie

Url:

<http://localhost:8080/api/v1.0/moviebooking/{moviename}/delete/{theatreName}>

Response

Movie Deleted Successfully

7. Update Status

Url:http://localhost:8080/api/v1.0/moviebooking/moviename/theatreName?status=BOOK_ASAP

Response :

Movie status updated to BOOK_ASAP

8. View Available Movies

Url : <http://localhost:8080/api/v1.0/moviebooking/all>

Response :

```
[
  {
    "movieName": "Manam",
    "theatreName": "GPR",
    "status": "BOOK_ASAP",
    "totalTickets": 10,
    "ticketsAvailable": 10,
    "ticketsBooked": 0
  },
  {
    "movieName": "shiva",
    "theatreName": "Mjr",
    "status": "AVAILABLE",
    "totalTickets": 10,
    "ticketsAvailable": 9,
    "ticketsBooked": 1
  }
]
```


9. Movie Booking

Url : <http://localhost:8080/api/v1.0/moviebooking/book>

Request Payload :

```
{
  "movieName": "shiva",
  "theatreName": "mjr",
  "numberOfTickets": 1,
  "seatNumbers": ["A5"]
}
```

Response:

Ticket booked successfully for seats: [A5]

10. Search By Movie Name

Url : <http://localhost:8080/api/v1.0/moviebooking/movies/search/shiva>

Response

```
[
  {
    "movieName": "Shiva",
    "theatreName": "Asian",
    "status": "AVAILABLE",
    "totalTickets": 10,
    "ticketsAvailable": 6,
    "ticketsBooked": 4,
    "seats": [
      {
        "seatNumber": "A1",
        "status": "BOOKED"
      },
      {
        "seatNumber": "A2",
        "status": "BOOKED"
      },

```

```
{
  "seatNumber": "A3",
  "status": "BOOKED"
},
{
  "seatNumber": "A4",
  "status": "BOOKED"
},
{
  "seatNumber": "A5",
  "status": "AVAILABLE"
},
{
  "seatNumber": "A6",
  "status": "AVAILABLE"
},
{
  "seatNumber": "A7",
  "status": "AVAILABLE"
},
{
  "seatNumber": "A8",
  "status": "AVAILABLE"
},
{
  "seatNumber": "A9",
  "status": "AVAILABLE"
},
{
  "seatNumber": "A10",
  "status": "AVAILABLE"
}
]
]
```

11. View Booking

Url : <http://localhost:8080/api/v1.0/moviebooking/view/booking>

Response :

```
[
  {
    "movieName": "Shiva",
    "theatreName": "Asian",
    "numberOfTickets": 2,
    "seatNumbers": "A1, A2",
    "status": "Confirmed"
  },
  {
    "movieName": "shiva",
    "theatreName": "Mjr",
    "numberOfTickets": 1,
    "seatNumbers": "A1",
    "status": "Confirmed"
  },
  {
    "movieName": "shiva",
    "theatreName": "Mjr",
    "numberOfTickets": 1,
    "seatNumbers": "A5",
    "status": "Confirmed"
  }
]
```

1. Register User

MovieBookingApp

Create an Account

Login Register

First Name

Only letters allowed

Last Name

Only letters allowed

Email

Invalid email format

Login ID

Must be 8-20 characters

Password

Must contain at least 8 characters, including uppercase, lowercase, number, and special character

Confirm Password

Contact Number

Must be exactly 10 digits

Register

Already have an account? [Login here](#)

2. Login Page

MovieBookingApp

Login Register

Welcome Back 🎬

Login ID

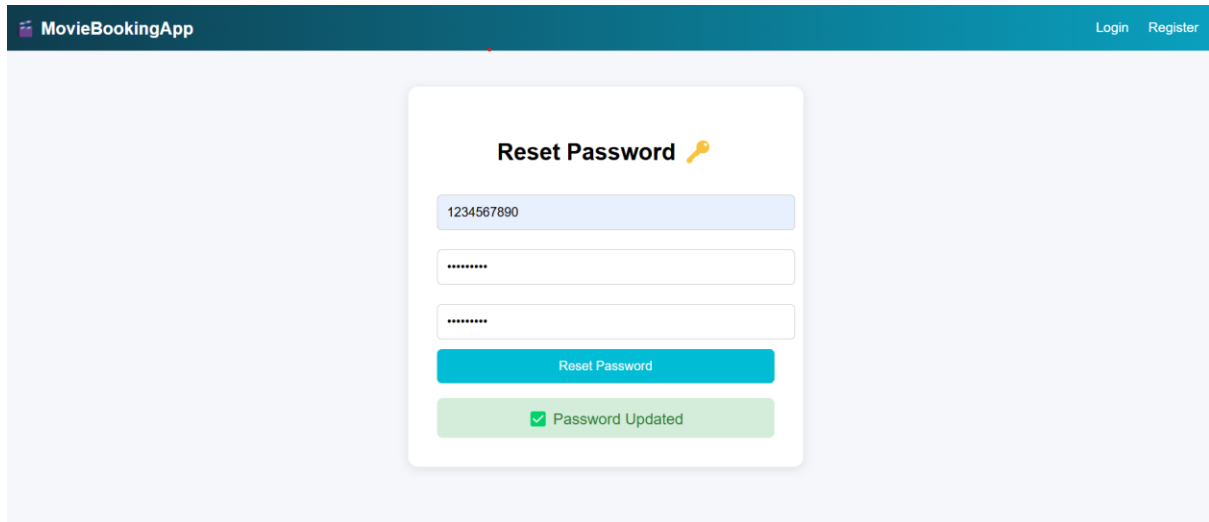
Password

Login

Don't have an account? [Register here](#)

[Forgot Password?](#)

3. Reset Password when user forgot their password



The screenshot shows the 'Reset Password' form in the MovieBookingApp. The form is centered on a light blue background. It has a title 'Reset Password' with a key icon. Below the title are three input fields: a text field containing '1234567890', a password field with masked characters, and another password field with masked characters. A blue 'Reset Password' button is below the input fields. At the bottom, a green message box says '✓ Password Updated'.

MovieBookingApp Login Register

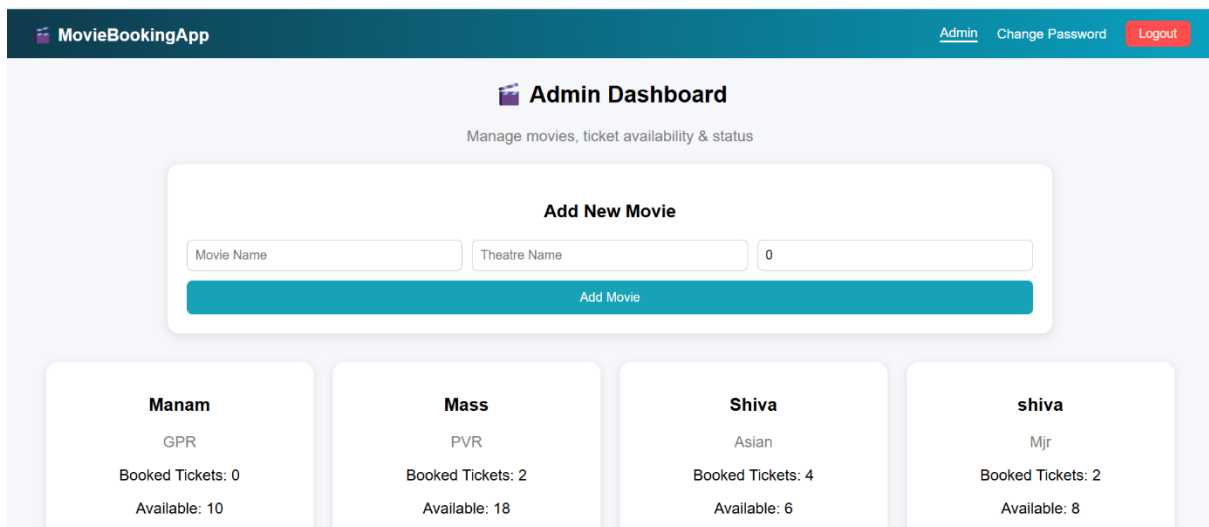
Reset Password

1234567890

Reset Password

✓ Password Updated

4. Admin Dashboard after login



The screenshot shows the 'Admin Dashboard' in the MovieBookingApp. The dashboard has a dark blue header with the app name and navigation links. The main content area is light blue and features an 'Add New Movie' form with three input fields and an 'Add Movie' button. Below the form are four movie cards, each showing the movie name, genre, booked tickets, and available tickets.

MovieBookingApp Admin Change Password Logout

Admin Dashboard

Manage movies, ticket availability & status

Add New Movie

Movie Name Theatre Name 0

Add Movie

Manam GPR Booked Tickets: 0 Available: 10	Mass PVR Booked Tickets: 2 Available: 18	Shiva Asian Booked Tickets: 4 Available: 6	shiva Mjr Booked Tickets: 2 Available: 8
---	--	--	--

5. Admin Dashboard After movie add

MovieBookingApp

[Admin](#) [Change Password](#) [Logout](#)

Admin Dashboard

Manage movies, ticket availability & status

Movie Added Successfully with Seats

Add New Movie

Bahubali

PVR

10

Add Movie

Bahubali

PVR

Booked Tickets: 0

Available: 20

Manam

GPR

Booked Tickets: 0

Available: 10

Mass

PVR

Booked Tickets: 2

Available: 18

Shiva

Asian

Booked Tickets: 4

Available: 6

6. User can change password after login

MovieBookingApp

[Home](#) [My Tickets](#) [Change Password](#) [Logout](#)

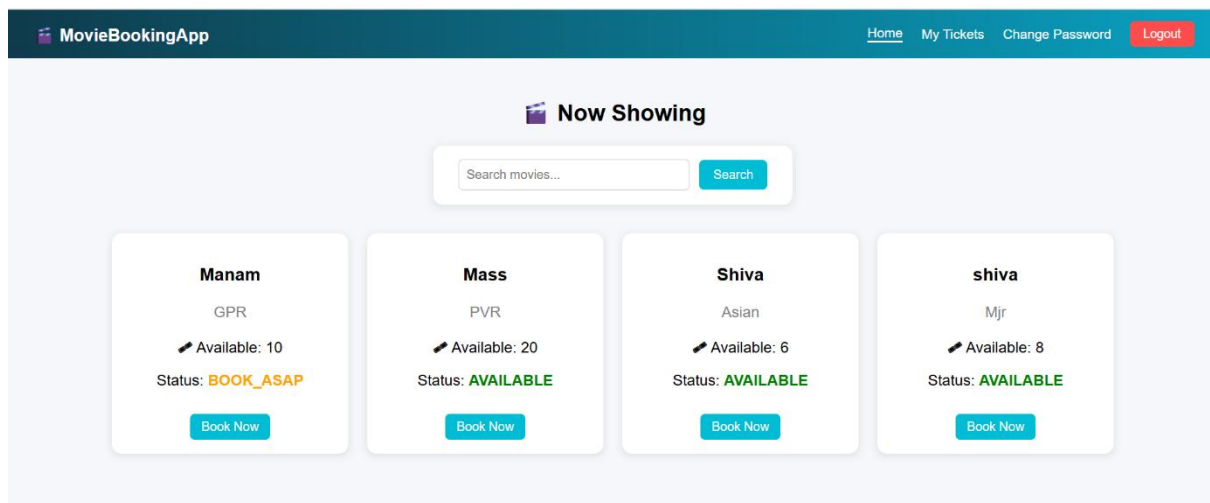
Reset Password 

Must contain at least 8 characters, including uppercase, lowercase, number, and special character

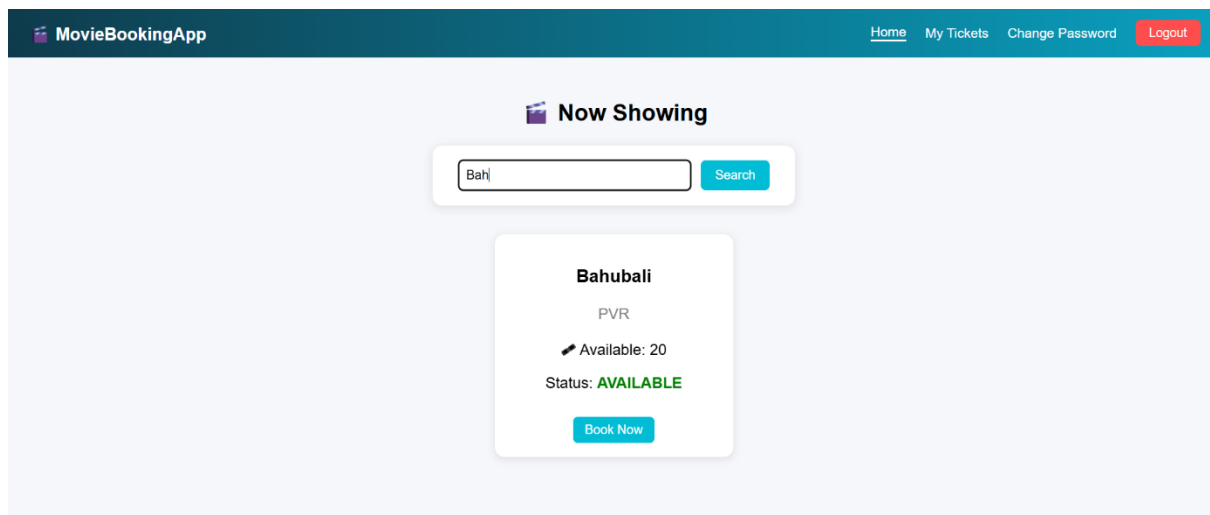
Passwords do not match

Reset Password

7. User Dashboard after login



8. User can search for particular movie



9. Seat layout after selecting movie

MovieBookingApp

HomeMy TicketsChange PasswordLogout

Select Your Seats

Mass – Pvr

No. of Tickets

1

A1A2A3A4A5A6A7A8A9A10

B1B2B3B4B5B6B7B8B9B10

Confirm Booking

10. Seat Selection and Ticket booking

MovieBookingApp

HomeMy TicketsChange PasswordLogout

Select Your Seats

Mass – Pvr

No. of Tickets

2

A1A2A3A4A5A6A7A8A9A10

B1B2B3B4B5B6B7B8B9B10

Confirm Booking

Ticket booked successfully for seats: [A1, A2]

Unit Testing - Backend

The screenshot shows an IDE window with the following components:

- Top Bar:** Displays the project name "Movie-Booking-System-App" and the current version control state.
- Project View:** Lists the files in the project, including "ketBookingController.java", "MovieController.java", "MovieService.java", and "MovieServiceImpl.java".
- Debug Console:** Shows the output of the test run. The status bar indicates "59 tests passed" and "59 tests total, 57 sec 252 ms".
- Test Results:** A list of test results for the "MovieService" class, showing the duration of each test. The tests are:
 - default pac 57 sec 252 ms
 - MovieServ 14 sec 79 ms
 - SeatRepos 2 sec 764 ms
 - TicketService 507 ms
 - AuthContr 4 sec 791 ms
 - MovieControlle 294 ms
 - ApplicationTests 19 ms
 - TicketBox 27 sec 971 ms
 - MovieRepository 742 ms
 - TicketRepositor 347 ms
 - testFindByLc 127 ms
 - testDeleteBy 220 ms
 - UserRepository 186 ms
 - Should return 43 ms
 - Should return 38 ms
 - Should find ut 67 ms
 - Should find ut 38 ms
 - UserServ 5 sec 552 ms
- Console Output:** The output shows the execution of the tests, including the "default pac" test which prints a large ASCII art logo. The output also shows the "Spring Boot" version (v3.2.5) and the "main" method of the "MovieService" class.

```
Terminal Local + -
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.168 s -- in com.moviebookingapp.api.domain.services.TicketServiceImplTest
[INFO] Running com.moviebookingapp.api.domain.services.UserServiceImplTest
[INFO] Tests run: 12, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.885 s -- in com.moviebookingapp.api.domain.services.UserServiceImplTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 59, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.3.0:jar (default-jar) @ Movie-Booking-System-App ---
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.3.0/plexus-utils-3.3.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.3.0/plexus-utils-3.3.0.pom (5.2 kB at 36 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-archiver/3.6.0/maven-archiver-3.6.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/maven-archiver/3.6.0/maven-archiver-3.6.0.pom (3.9 kB at 26 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-io/3.4.0/plexus-io-3.4.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-io/3.4.0/plexus-io-3.4.0.pom (6.0 kB at 48 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-archiver/4.4.0/plexus-archiver-4.4.0.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-archiver/4.4.0/plexus-archiver-4.4.0.pom (6.3 kB at 54 kB/s)
)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-compress/1.21/commons-compress-1.21.pom
```

Test Reports :

Movie-Booking-System-App > com.moviebookingapp.api.domain.services.impl

com.moviebookingapp.api.domain.services.impl

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
MovieServiceImpl		92%		82%	5	23	5	84	1	11	0	1
UserServiceImpl		96%		92%	2	16	1	43	1	9	0	1
TicketServiceImpl		100%		91%	1	19	0	109	0	13	0	1
Total	32 of 952	96%	6 of 49	87%	8	58	6	236	2	33	0	3

Created

Movie-Booking-System-App > com.moviebookingapp.api.domain.controllers

com.moviebookingapp.api.domain.controllers

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
AuthController		59%		37%	4	9	11	30	0	5	0	1
TicketBookingController		100%		n/a	0	4	0	19	0	4	0	1
MovieController		100%		n/a	0	5	0	19	0	5	0	1
Total	59 of 299	80%	5 of 8	37%	4	18	11	68	0	14	0	3

Created

Unit Testing - Frontend

EXPLORER

MOVIE-BOOKING-UI

.angular

.vscode

node_modules

public

src

app

admin

auth

forgot-password

login

register

reset-password

constants

core

shared

user

booking

booking.css

booking.html

booking.spec.ts

booking.ts

dashboard

my-bookings

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

✓ should create 937ms

✓ should show error if form is invalid 302ms

✓ movie-booking-ui src/app/admin/dashboard/dashboard.spec.ts (7 tests) 1743ms

✓ should create 779ms

✓ movie-booking-ui src/app/auth/reset-password/reset-password.spec.ts (1 test) 6ms

✓ movie-booking-ui src/app/user/my-bookings/my-bookings.spec.ts (2 tests) 167ms

✓ movie-booking-ui src/app/app.spec.ts (1 test) 178ms

✓ movie-booking-ui src/app/shared/navbar/navbar.spec.ts (2 tests) 369ms

✓ should create 334ms

Test Files 10 passed (10)

Tests 37 passed (37)

Start at 13:05:11

Duration 41.06s (transform 5.20s, setup 42.60s, import 11.57s, tests 9.80s, environment 241.09s)

The End